

# Pixel Sheet Converter 1.2

Conversione percettiva di immagini in matrici cromatiche, PNG e XLSX

Documento tecnico esplicativo

13 agosto 2026

## Scopo del documento

Il software trasforma un'immagine continua in una matrice discreta: ogni pixel dell'immagine ridimensionata diventa un indice di palette e, nel foglio XLSX, una cella quadrata con riempimento statico. La versione 1.2 introduce metriche percettive, otto metodi di dithering, il calcolo della dimensione fisica per pen plotter e un'interfaccia Windows 2000 modernizzata. Questo documento descrive formule, motivazioni, compromessi e corrispondenza con il codice.

## Indice

|                                                    |          |
|----------------------------------------------------|----------|
| <b>1 Pipeline completa</b>                         | <b>2</b> |
| <b>2 Lettura, ridimensionamento e scala fisica</b> | <b>2</b> |
| 2.1 Normalizzazione e ricampionamento . . . . .    | 2        |
| 2.2 Dal passo pixel ai millimetri . . . . .        | 2        |
| <b>3 Palette e rappresentazione discreta</b>       | <b>3</b> |
| <b>4 Metriche di confronto colore</b>              | <b>3</b> |
| 4.1 RGB classico . . . . .                         | 3        |
| 4.2 RGB lineare . . . . .                          | 3        |
| 4.3 OKLab percettivo . . . . .                     | 3        |
| 4.4 CIELAB e Delta E 2000 . . . . .                | 4        |
| <b>5 Quantizzazione e dithering</b>                | <b>4</b> |
| 5.1 Diffusione dell'errore . . . . .               | 4        |
| 5.2 Bayer 4x4 . . . . .                            | 4        |
| <b>6 Esportazione PNG e XLSX</b>                   | <b>5</b> |
| <b>7 Statistiche, prestazioni e limiti</b>         | <b>5</b> |
| <b>8 Architettura del software desktop</b>         | <b>5</b> |
| <b>9 Aggiornamenti portatili e distribuzione</b>   | <b>6</b> |
| <b>10 Guida alla scelta</b>                        | <b>6</b> |
| <b>11 Verifiche</b>                                | <b>6</b> |
| <b>12 Limiti e sviluppi futuri</b>                 | <b>6</b> |
| <b>13 Conclusione</b>                              | <b>6</b> |

# 1 Pipeline completa

La pipeline corrente è composta dalle seguenti fasi:

1. lettura e normalizzazione dell'immagine in sRGB a tre canali;
2. ridimensionamento, mantenendo automaticamente il rapporto d'aspetto;
3. definizione di una palette incorporata o personalizzata;
4. trasformazione dei colori nello spazio scelto;
5. assegnazione del colore di palette più vicino;
6. eventuale dithering ordinato o diffusione dell'errore;
7. produzione dell'anteprima PNG e delle statistiche;
8. esportazione XLSX con celle quadrate e riempimenti statici;
9. verifica dell'integrità del contenitore XLSX.

Per l'immagine di riferimento, il passaggio da  $1086 \times 1448$  a  $543 \times 724$  produce

$$543 \cdot 724 = 393\,132 \text{ celle-pixel.}$$

Il rapporto 3 : 4 è preservato e ogni dimensione lineare viene dimezzata.

## 2 Lettura, ridimensionamento e scala fisica

### 2.1 Normalizzazione e ricampionamento

#### Operazione

```
source = Image.open(path)
source.load()
source = source.convert("RGB")
size = target_size(source.size, width)
resized = source.resize(size, filter)
```

#### Motivazione

La conversione esplicita in RGB elimina differenze fra immagini in scala di grigi, RGBA o altri modi PIL. Lanczos è il metodo predefinito perché conserva bene i dettagli nella riduzione. Bicubico e bilineare sono più morbidi; nearest-neighbor preserva invece i blocchi netti della pixel art.

Con larghezza richiesta  $W$  e sorgente  $W_s \times H_s$ , l'altezza è

$$H = \max \left( 1, \text{round} \left( H_s \frac{W}{W_s} \right) \right).$$

Il programma impone inoltre un limite di sicurezza di due milioni di celle.

### 2.2 Dal passo pixel ai millimetri

Il parametro *passo pixel*  $p$  non modifica l'immagine digitale; descrive la distanza fisica fra due campioni consecutivi nel successivo workflow CNC o pen plotter. Le dimensioni teoriche sono

$$L_x = Wp, \quad L_y = Hp.$$

Per esempio,  $543 \times 724$  pixel con  $p = 0,5$  mm corrispondono a  $271,5 \times 362$  mm. Il diametro della penna, la sovrapposizione e la calibrazione meccanica rimangono responsabilità del workflow Grasshopper/CNC.

### 3 Palette e rappresentazione discreta

Una palette  $P = \{\mathbf{c}_0, \dots, \mathbf{c}_{K-1}\}$  contiene colori sRGB espressi come triplette 0–255. Può essere una palette incorporata (RGB, RGB con bianco e nero, CMY, grigi) oppure una lista personalizzata di codici #RRGGBB. Nel caso primario:

| Indice | Campione                                                                          | Valore sRGB |
|--------|-----------------------------------------------------------------------------------|-------------|
| 0      |  | #FF0000     |
| 1      |  | #00FF00     |
| 2      |  | #0000FF     |

Il risultato fondamentale non è una bitmap incorporata, ma una matrice  $Q \in \{0, \dots, K-1\}^{H \times W}$ . Lo stesso  $Q$  genera anteprima, conteggi, PNG e celle XLSX; questo evita discrepanze tra gli output.

### 4 Metriche di confronto colore

Il colore assegnato al pixel  $\mathbf{p}$  è

$$i^* = \arg \min_{0 \leq i < K} d(T(\mathbf{p}), T(\mathbf{c}_i)),$$

dove  $T$  dipende dalla metrica scelta. Sono implementate quattro modalità.

#### 4.1 RGB classico

I canali vengono normalizzati in  $[0, 1]$  e confrontati con distanza euclidea:

$$d_{RGB}^2 = \sum_{j \in \{r, g, b\}} (p_j - c_j)^2.$$

È rapido e intuitivo, ma opera su valori sRGB codificati con gamma; uguali distanze numeriche non corrispondono a uguali differenze visive.

#### 4.2 RGB lineare

Prima del confronto, ciascun canale sRGB normalizzato  $u$  viene linearizzato:

$$u_{lin} = \begin{cases} u/12,92, & u \leq 0,04045, \\ ((u + 0,055)/1,055)^{2,4}, & u > 0,04045. \end{cases}$$

La distanza euclidea viene quindi calcolata sull'energia luminosa lineare. È più fisicamente coerente dell'RGB classico, pur non essendo percettivamente uniforme.

#### 4.3 OKLab percettivo

L'RGB lineare viene trasformato prima nei segnali intermedi  $l, m, s$ :

$$\begin{bmatrix} l \\ m \\ s \end{bmatrix} = \begin{bmatrix} 0,4122214708 & 0,5363325363 & 0,0514459929 \\ 0,2119034982 & 0,6806995451 & 0,1073969566 \\ 0,0883024619 & 0,2817188376 & 0,6299787005 \end{bmatrix} \begin{bmatrix} r \\ g \\ b \end{bmatrix}.$$

Dopo la radice cubica componente per componente, una seconda matrice produce  $L, a, b$ . La distanza usata è euclidea in OKLab. È il valore predefinito perché offre un buon equilibrio fra somiglianza visiva e costo computazionale.

#### 4.4 CIELAB e Delta E 2000

L'RGB lineare viene trasformato in XYZ D65, normalizzato rispetto al bianco di riferimento e convertito in CIELAB. La distanza  $\Delta E_{00}$  corregge le differenze di luminosità, croma e tinta:

$$\Delta E_{00} = \sqrt{\left(\frac{\Delta L'}{S_L}\right)^2 + \left(\frac{\Delta C'}{S_C}\right)^2 + \left(\frac{\Delta H'}{S_H}\right)^2} + R_T \frac{\Delta C'}{S_C} \frac{\Delta H'}{S_H}.$$

I fattori  $S_L, S_C, S_H$  e il termine di rotazione  $R_T$  modellano la diversa sensibilità umana nelle regioni dello spazio cromatico. È la scelta più raffinata disponibile, ma durante la diffusione dell'errore richiede più calcoli per ogni pixel.

| Metrica                | Costo relativo | Vantaggio             | Uso consigliato      |
|------------------------|----------------|-----------------------|----------------------|
| RGB classico           | minimo         | velocità              | prove rapide         |
| RGB lineare            | basso          | luminosità fisica     | resa tonale semplice |
| OKLab                  | medio          | accordo percettivo    | scelta generale      |
| CIELAB $\Delta E_{00}$ | alto           | confronto dettagliato | output finale        |

### 5 Quantizzazione e dithering

Senza dithering, ogni pixel viene assegnato indipendentemente al colore più vicino. È il metodo più veloce e produce regioni nette, ma perde sfumature.

#### 5.1 Diffusione dell'errore

Dopo l'assegnazione, l'errore resta calcolato nello spazio RGB di lavoro:

$$\mathbf{e} = \mathbf{p} - \mathbf{c}_{i^*}.$$

Esso viene distribuito ai pixel futuri secondo un kernel. Nel percorso serpentino la direzione è invertita nelle righe dispari, riducendo le trame orientate. Il buffer viene limitato a  $[0, 255]$  dopo ogni aggiornamento.

| Metodo              | Supporto  | Carattere visivo     | Considerazioni           |
|---------------------|-----------|----------------------|--------------------------|
| Floyd–Steinberg     | 4 vicini  | dettagliato          | standard equilibrato     |
| Floyd serpentino    | 4 vicini  | meno direzionale     | predefinito              |
| Atkinson            | 6 vicini  | chiaro e contrastato | diffonde 6/8 dell'errore |
| Sierra Lite         | 3 vicini  | leggero              | adatto a grandi matrici  |
| Stucki              | 12 vicini | morbido              | diffusione ampia         |
| Jarvis–Judice–Ninke | 12 vicini | gradazioni regolari  | kernel ampio             |

Per Floyd–Steinberg i pesi sono  $7/16, 3/16, 5/16, 1/16$  e sommano a uno. Sierra Lite usa  $2/4, 1/4, 1/4$ . Stucki normalizza il kernel su 42; JJN su 48.

#### 5.2 Bayer 4x4

Il dithering ordinato non propaga l'errore. A ogni pixel viene aggiunta una soglia periodica derivata dalla matrice

$$B = \begin{bmatrix} 0 & 8 & 2 & 10 \\ 12 & 4 & 14 & 6 \\ 3 & 11 & 1 & 9 \\ 15 & 7 & 13 & 5 \end{bmatrix}.$$

Nel programma l'offset è  $(B/16 - 1/2) \cdot 64$ . Il risultato è deterministico, veloce e regolare, ma può mostrare la trama 4x4.

## 6 Esportazione PNG e XLSX

Il PNG viene costruito sostituendo ogni indice con la sua tripletta di palette. Per XLSX, ogni indice è scritto come valore numerico nella cella corrispondente; il valore viene nascosto e il colore è applicato come riempimento statico.

```
for y in range(height):
    start = 0
    color = indices[y, 0]
    for x in range(1, width + 1):
        if x == width or indices[y, x] != color:
            write_run(y, start, x - 1, color)
            start = x
        if x < width:
            color = indices[y, x]
```

Il raggruppamento delle sequenze orizzontali riduce le operazioni di stile. I riempimenti statici, rispetto alla formattazione condizionale sperimentata inizialmente, sono importati in modo più affidabile da Google Fogli. Righe e colonne sono configurate per apparire quadrate e la griglia viene nascosta. Al termine il file viene riaperto come archivio ZIP e verificato.

## 7 Statistiche, prestazioni e limiti

I conteggi derivano direttamente dalla matrice degli indici:

$$n_i = \#\{(x, y) : Q_{y,x} = i\}, \quad f_i = \frac{n_i}{WH}.$$

Con  $K$  colori, la quantizzazione senza dithering richiede  $\Theta(WHK)$  operazioni. I kernel di diffusione aggiungono un numero costante di aggiornamenti per pixel, quindi l'ordine asintotico non cambia. La memoria è  $\Theta(WH)$  per buffer RGB e matrice degli indici.

In pratica,  $543 \times 724$  è un compromesso già impegnativo per XLSX e pen plotter. Una matrice da 177 689 punti per un solo canale può generare tempi e percorsi meccanici molto elevati; il diradamento va scelto in funzione del passo, del diametro della penna e della scala finale, non solo dell'aspetto a schermo.

## 8 Architettura del software desktop

Il software separa quattro responsabilità:

- `converter.py`: trasformazioni cromatiche, dithering e matrice;
- `exporters.py`: serializzazione PNG/XLSX e verifica;
- `gui.py`: controlli, anteprime, statistiche e attività in thread;
- `updater.py`: interrogazione manuale dell'ultima GitHub Release.

L'interfaccia 1.2 adotta un linguaggio Windows 2000 modernizzato: Tahoma, grigio #D4D0C8, intestazione blu #000080, controlli compatti, menu, toolbar e barra di stato. Questa scelta è esclusivamente presentazionale: non cambia la matrice risultante. La cornice esterna resta nativa di Windows, preservando DPI, ridimensionamento e accessibilità.

Le conversioni e le esportazioni lunghe sono eseguite in thread secondari; gli aggiornamenti grafici transitano attraverso una coda elaborata dal thread Tk principale. Ciò evita di bloccare l'interfaccia durante operazioni costose.

## 9 Aggiornamenti portatili e distribuzione

Il programma non si sovrascrive mentre è in esecuzione. Il pulsante *Controlla aggiornamenti* legge l'ultima release pubblica, confronta il numero di versione e apre il download dello ZIP. L'utente chiude il programma, estrae la nuova cartella e conserva temporaneamente quella precedente.

Una GitHub Action associata ai tag `v*` esegue test su Windows, costruisce l'applicazione con PyInstaller, comprime la cartella portatile e allega `PixelSheetConverter-Windows-x64.zip` alla Release. Il sistema è reversibile e non richiede privilegi amministrativi o installer.

## 10 Guida alla scelta

| Obiettivo          | Metrica           | Dithering           | Nota               |
|--------------------|-------------------|---------------------|--------------------|
| Anteprima veloce   | RGB classico      | Nessuno/Sierra Lite | costo minimo       |
| Resa generale      | OKLab             | Floyd serpentino    | consigliata        |
| Gradazioni morbide | OKLab             | Stucki o JJN        | più lenta          |
| Trama regolare     | RGB lineare/OKLab | Bayer 4x4           | pattern visibile   |
| Confronto fine     | $\Delta E_{00}$   | Floyd serpentino    | costo massimo      |
| Pixel art          | RGB classico      | Nessuno + nearest   | conserva i blocchi |

## 11 Verifiche

La suite automatica controlla mantenimento delle proporzioni, dimensioni del risultato, validità degli indici, conteggi, esportazione PNG, integrità ZIP dell'XLSX e presenza degli stili RGB. Un test attraversa tutte le quattro metriche con i nuovi dithering Sierra Lite, Stucki e JJN. La pipeline GitHub esegue i test su Windows prima della pubblicazione del pacchetto.

## 12 Limiti e sviluppi futuri

- Il dithering seleziona il colore con una metrica eventualmente percettiva, ma diffonde ancora il vettore d'errore nel buffer RGB: è una scelta stabile e comprensibile, non una simulazione completa della visione.
- La dimensione fisica è teorica; non compensa backlash, diametro penna, deformazioni del supporto o calibrazione degli assi.
- XLSX non è un formato ideale per milioni di celle: oltre una certa scala conviene usare PNG o dati geometrici dedicati.
- Le palette molto ampie aumentano linearmente il costo del confronto.
- Il controllo aggiornamenti è manuale e richiede una connessione; le immagini restano elaborate localmente.

## 13 Conclusione

Pixel Sheet Converter 1.2 non si limita a sostituire un pixel con il primario RGB numericamente più vicino. Offre una catena controllabile che separa ricampionamento, spazio di confronto, dithering, scala fisica e formato di uscita. OKLab costituisce il compromesso predefinito per una lettura più vicina alla percezione umana; CIEDE2000 fornisce un'alternativa più rigorosa; i diversi kernel permettono di controllare trama e costo. La matrice unica degli indici mantiene coerenti anteprima, statistiche, PNG e XLSX, mentre la separazione tra motore, interfaccia ed esportazione rende l'applicazione verificabile e aggiornabile senza installazione.

## Riferimenti essenziali

- IEC 61966-2-1, *Multimedia systems and equipment - Colour measurement and management - sRGB*.
- G. Sharma, W. Wu, E. N. Dalal, “The CIEDE2000 Color-Difference Formula”, *Color Research & Application*, 2005.
- B. Ottosson, *A perceptual color space for image processing*, 2020, <https://bottosson.github.io/posts/oklab/>.
- R. W. Floyd, L. Steinberg, “An Adaptive Algorithm for Spatial Greyscale”, *Proceedings of the SID*, 1976.
- ECMA-376, *Office Open XML File Formats*.